

PROMPT — Módulo de Cierre de Caja y Auditoría de Turno

Pegar completo en Replit

Necesito implementar un módulo completo de Caja y Turnos. Cada área del hotel (Recepción, Restaurante, SPA) tiene su propia caja. La Recepción opera en 3 turnos diarios, el Restaurante cierra 2 cajas por día y el SPA 1 caja por día. La configuración de turnos por área debe ser editable para adaptarse a futuras necesidades.

PARTE 1 — Schema de base de datos

Agregar las siguientes tablas en `shared/schema.ts` :

Tabla `cashRegisterConfigs` — Configuración de cajas por área

```
export const cashRegisterConfigs = pgTable("cash_register_configs", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  area: text("area").notNull().unique(),
  // Valores: "reception", "restaurant", "spa", "events"
  areaLabel: text("area_label").notNull(),
  // Etiqueta visible: "Recepción", "Restaurante", "SPA"
  shiftsPerDay: integer("shifts_per_day").notNull().default(1),
  // Cantidad de turnos/cierres por día: Recepción=3, Restaurante=2, SPA=1
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});
```

Tabla `cashShifts` — Turnos de caja

```
export const cashShifts = pgTable("cash_shifts", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  area: text("area").notNull(),
  // "reception", "restaurant", "spa", "events"
  shiftNumber: integer("shift_number").notNull(),
```

```

// Número de turno del día: 1, 2 o 3
openedBy: text("opened_by").notNull(),
// Nombre del usuario que abre el turno
closedBy: text("closed_by"),
// Nombre del usuario que cierra el turno
openedAt: timestamp("opened_at").notNull().defaultNow(),
closedAt: timestamp("closed_at"),
status: text("status").notNull().default("open"),
// "open" | "closed"
notes: text("notes"),
// Observaciones al abrir o cerrar
createdAt: timestamp("created_at").defaultNow(),
});

```

Tabla `cashMovements` — Movimientos registrados en el turno

```

export const cashMovements = pgTable("cash_movements", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  shiftId: varchar("shift_id").notNull().references(() => cashShifts.id),
  area: text("area").notNull(),

  // Origen del movimiento
  sourceType: text("source_type").notNull(),
  // "reservation", "restaurant_order", "spa_account", "event", "manual"
  sourceId: varchar("source_id"),
  // ID de la reserva, orden, cuenta SPA o evento
  sourceLabel: text("source_label"),
  // Descripción legible: "Reserva RES-0042 - García Juan", "Mesa 5 - Orden #123"

  // Detalle del movimiento
  paymentMethod: text("payment_method").notNull(),
  // "cash" | "debit_card" | "credit_card" | "transfer" | "mercadopago" | "current"
  amount: numeric("amount", { precision: 10, scale: 2 }).notNull(),
  movementType: text("movement_type").notNull().default("income"),
  // "income" (ingreso) | "expense" (egreso/devolución)

  receiptType: text("receipt_type"),
  // "ticket" | "factura_a" | "factura_b" | "factura_c"

  registeredBy: text("registered_by"),
  createdAt: timestamp("created_at").defaultNow(),
});

```

Tabla `cashClosingSummaries` — Resumen del cierre

```

export const cashClosingSummaries = pgTable("cash_closing_summaries", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  shiftId: varchar("shift_id").notNull().references(() => cashShifts.id),
  area: text("area").notNull(),

  // Totales por método de pago al cierre
  totalCash: numeric("total_cash", { precision: 10, scale: 2 }).default("0"),
  totalDebitCard: numeric("total_debit_card", { precision: 10, scale: 2 }).default("0"),
  totalCreditCard: numeric("total_credit_card", { precision: 10, scale: 2 }).default("0"),
  totalTransfer: numeric("total_transfer", { precision: 10, scale: 2 }).default("0"),
  totalMercadopago: numeric("total_mercadopago", { precision: 10, scale: 2 }).default("0"),
  totalCurrentAccount: numeric("total_current_account", { precision: 10, scale: 2 }).default("0"),
  totalRoomCharge: numeric("total_room_charge", { precision: 10, scale: 2 }).default("0"),
  totalGeneral: numeric("total_general", { precision: 10, scale: 2 }).default("0"),

  // Cantidad de transacciones
  transactionCount: integer("transaction_count").default(0),

  closedAt: timestamp("closed_at").defaultNow(),
  closedBy: text("closed_by"),
  notes: text("notes"),
});

```

PARTE 2 — Endpoints en `server/routes.ts` y métodos en `server/db-storage.ts`

```

// Configuración
GET    /api/cash/configs                → Listar configuraciones de cajas
PATCH /api/cash/configs/:area          → Editar config de un área (shiftsPerD

// Turnos
GET    /api/cash/shifts?area=&status=  → Listar turnos (filtrar por área y/o
GET    /api/cash/shifts/current?area=  → Turno actualmente abierto de un área
POST   /api/cash/shifts/open           → Abrir nuevo turno
POST   /api/cash/shifts/:id/close      → Cerrar turno (genera resumen automát
GET    /api/cash/shifts/:id           → Detalle de un turno con sus movimien

// Movimientos
GET    /api/cash/movements?shiftId=    → Listar movimientos de un turno
POST   /api/cash/movements             → Registrar movimiento manual

// Resumen
GET    /api/cash/summary?area=&from=&to= → Resumen histórico de cierres por áre

```

Lógica del endpoint `POST /api/cash/shifts/open`

1. Verificar que no haya un turno abierto para esa área.
2. Calcular el número de turno del día (cuántos turnos ya se abrieron hoy para esa área + 1).
3. Crear el registro en `cashShifts` con `status: "open"`.
4. Retornar el turno creado.

Lógica del endpoint `POST /api/cash/shifts/:id/close`

1. Verificar que el turno existe y está en estado `open`.
2. Obtener todos los `cashMovements` del turno.
3. Calcular totales por método de pago.
4. Crear registro en `cashClosingSummaries` con los totales.
5. Actualizar `cashShifts` con `status: "closed"`, `closedAt` y `closedBy`.
6. Retornar el resumen del cierre.

Lógica de registro automático de movimientos

Cuando se registra un pago en cualquier módulo del sistema (reserva, restaurante, SPA, evento), el sistema debe buscar el turno abierto del área correspondiente y crear automáticamente un `cashMovement`.

Modificar los siguientes endpoints existentes para que, después de crear el pago, llamen a un método `registerCashMovement()`:

- `POST /api/payments` (área: "reception")
- `POST /api/restaurant/orders/:id/close` (área: "restaurant")
- `POST /api/spa/accounts/:id/close` (área: "spa")
- `POST /api/events/:id/close` (área: "events")

Si no hay turno abierto para el área, el movimiento igual se registra pero con `shiftId: null` y se marca como "sin turno asignado" en `sourceLabel`.

PARTE 3 — Frontend: Módulo de Caja

Nueva página `client/src/pages/cash-register.tsx` accesible desde el sidebar bajo "Caja" con ícono `Wallet`.

Estructura de la página (tabs por área)

Tabs: **Recepción** | **Restaurante** | **SPA** | **Historial**

Tab de área (igual para Recepción, Restaurante y SPA)

Estado del turno actual:

Si NO hay turno abierto:

- Mensaje: "No hay turno abierto para [Área]"
- Botón verde "Abrir Turno" → abre un dialog que pide:
 - Nombre del responsable (texto libre)
 - Observaciones opcionales
 - Botón "Confirmar apertura"

Si HAY turno abierto:

- Badge verde "Turno abierto" con la hora de apertura y el nombre del responsable
- Turno N° del día (ej: "Turno 2 de 3")

Movimientos del turno actual:

Tabla con columnas: Hora | Origen | Descripción | Método de pago | Tipo | Monto

- Origen muestra el área/módulo con un badge de color
- Tipo: "Ingreso" en verde / "Egreso" en rojo
- Monto con formato de moneda

Botón "Registrar movimiento manual" → abre dialog con:

- Tipo: Ingreso / Egreso
- Descripción libre
- Método de pago (dropdown)

- Monto
- Comprobante (opcional)

Resumen parcial del turno (visible todo el tiempo):

Tabla de totales parciales por método:

Método	Transacciones	Total
Efectivo	5	\$45.000
Débito	3	\$28.500
...	...	...
TOTAL GENERAL	8	\$73.500

Botón "Cerrar Turno" (rojo, visible solo si hay turno abierto):

- Abre dialog de cierre con:
 - Resumen completo de movimientos del turno
 - Totales por método de pago
 - Campo "Cerrado por" (nombre del responsable)
 - Campo "Observaciones" (opcional)
 - Botón "Confirmar cierre" → ejecuta el cierre y muestra el resumen final
 - El resumen final tiene botón "Imprimir" que abre `window.print()`

Tab Historial

Filtros: Área (todas / recepción / restaurante / SPA) + rango de fechas

Tabla de cierres históricos: Columnas: Fecha | Área | Turno | Abierto por | Cerrado por | Hora apertura | Hora cierre | Total general | Acciones

Botón "Ver detalle" en cada fila → abre modal con el detalle completo del turno (todos los movimientos + resumen por método).

Botón "Imprimir" en el modal de detalle.

Configuración de cajas (accesible desde Administración)

En `client/src/pages/administration.tsx` agregar una sección "Configuración de Cajas" con:

- Lista de áreas con su configuración actual (turnos por día, estado activo/inactivo)
 - Botón "Editar" por área que permite cambiar `shiftsPerDay` y `areaLabel`
 - Esto permite que si el hotel cambia o un nuevo hotel se configura diferente, se ajuste sin tocar código
-

PARTE 4 — Datos seed iniciales

En `server/seed.ts` agregar la inserción de las configuraciones iniciales de cajas:

```
// Configuraciones de caja iniciales
await db.insert(cashRegisterConfigs).values([
  { id: randomUUID(), area: "reception", areaLabel: "Recepción", shiftsPerDay: 3,
  { id: randomUUID(), area: "restaurant", areaLabel: "Restaurante", shiftsPerDay:
  { id: randomUUID(), area: "spa", areaLabel: "SPA", shiftsPerDay: 1, isActive: t
  { id: randomUUID(), area: "events", areaLabel: "Eventos", shiftsPerDay: 1, isAc
]).onConflictDoNothing();
```

Sidebar y rutas

En `client/src/components/app-sidebar.tsx` agregar "Caja" con ícono `Wallet` de `lucide-react`.

En `client/src/App.tsx` agregar:

```
<Route path="/cash-register" component={CashRegister} />
```

Notas para Replit

- Implementar en orden: Parte 1 (schema) → Parte 2 (endpoints) → Parte 3 (frontend) → Parte 4 (seed)
- El registro automático de movimientos de caja al cerrar órdenes/cuentas es

importante — no omitirlo

- “Cierre de habitación” (`room_charge`) es un método de pago válido que mueve dinero entre áreas — en la caja de recepción aparece como ingreso cuando se cierra la habitación
- El formato de moneda en toda la interfaz es ARS sin decimales (ej: \$45.000)
- Si no hay turno abierto al registrar un pago, el sistema no debe bloquearlo — solo registra el movimiento sin turno asignado
- Ejecutar `npm run db:push` después de agregar las tablas al schema
- Todos los textos en español
- Verificar que compila sin errores TypeScript al finalizar